implement your own encryption schema for passwords, or even worry about sessions. In our case while working with the PHP client library, you start the authentication procedure by calling the Facebook object's require_login method. Your users are then redirected to Facebook's login page (https://login.facebook.com/login.php), which is passed with your API key, and the user is given a session key and redirected to your callback page. When the user enters the application for the first time, he is asked to accept the terms of service for your application.

Instead of logging into Facebook every time you want to update the data to use some sort of scheduled task, you can do so with an infinite session key. The process to get your infinite key is however, a bit more convoluted. After you've made your application, create a new page (`infinite_key.php`) on your server, i.e., your callback domain. This will create a new Facebook object and echo your session_key:

```
<?php
$facebook_config['debug'] = false;
$facebook_config['api_key'] = '<your_api_key>';
$facebook_config['secret_key'] = '<your_secret_key>';
require_once('<path_to_api>/facebook.php');
$facebook = new Facebook($facebook_config['api_key'],
$facebook_config['secret']);
$user = $facebook->require_login();
$infinate_key = $facebook->api_client->session_key;
echo($infinate_key);
?>
```

Now log out of Facebook, clear your Facebook cookies and browser cache, and then go to the page you just created on your server. Once you've logged on, you should see the infinite key that you can then use in your code. You can now use your own UID and the key just obtained in other code to perform anything that needs to happen on a regular basis with the set_user function in the facebook object:

```
<?php
...
$uid = '<your_uid>';
$key = '<your infinite key>';
$facebook->set_user($uid, $key);
// other code
?>
```